

Edge cases (empty vs. missing) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/env_config_practice.py`, under its `# Edge cases` section — the same file also holds the [Intermediate exercise](#), under an `# Intermediate` section. Run it with `cd practice && python env_config_practice.py`.

Builds on: the `# Intermediate` section of that same file — you are testing *that* loader against the two tricky `.env` states, so keep both sections in the one file. The `MissingConfigError` used below is the same class the [Build Task](#) ends up with in `practice/build_task/exceptions.py`.

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and try it yourself](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and try it yourself

Basic Version

There are two different kinds of "empty" here:

1. The `.env` file exists but has nothing written in it.
2. The `.env` file has `SOME_KEY=` — the key is there, but nothing comes after the `=`.

For each case, your config loader has to decide: should this count as "missing" (raise an error), or "empty but okay"? You get to decide, and should be able to explain why.

Try both cases yourself:

- Make a totally empty `.env` file (no text at all), run your loader, see what happens.
- Make a `.env` with `OPENAI_API_KEY=` (equals sign, nothing after it), run your loader, see what happens.

Check: does `os.getenv("OPENAI_API_KEY")` give you `None`, or does it give you `""` (an empty piece of text)? That difference is the key to this whole exercise.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Python's `os.getenv()` already treats these two cases differently, on its own — spotting that is the whole exercise:

```
# practice/env_config_practice.py – Edge cases section
os.getenv("OPENAI_API_KEY") # key never set at all      -> returns None
os.getenv("OPENAI_API_KEY") # file has "OPENAI_API_KEY=" -> returns ""
```

`None` and `""` are two different values in Python. `None` means "this key was never set." `""` (an empty string) means "this key was set, but to nothing." Your `require_env()` has to decide whether *both* of those should raise `MissingConfigError`, or only the first.

Test both situations against your own `require_env()` (or equivalent) from the Build Task, and inspect the value precisely instead of guessing:

```
# practice/env_config_practice.py – Edge cases section
value = os.getenv("OPENAI_API_KEY")
print(repr(value)) # None -> prints: None
                  # ""  -> prints: ''
print(type(value)) # <class 'NoneType'> or <class 'str'>
```

Using `repr()` (instead of a plain `print(value)`) shows you the quotes around a string, so you can't mistake an empty string `''` for anything else.

Difference between Basic and Intermediate: Basic names the two situations in plain English and just says "try it and look." Intermediate shows exactly what `os.getenv()` returns for each one, in real code, and how to inspect it precisely with `repr()` and `type()` so you're certain what you're looking at — this is the depth the exercise's Solution is written at.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
test 1: empty .env file
    run load_config()
    watch: does it raise an error, or does it keep going quietly?
    write down what happened, and whether that's what you want

test 2: .env has "OPENAI_API_KEY=" (nothing after the =)
    run load_config()
    watch: is the value None, or is it "" (empty text)?
    decide: should empty text count as "missing"? write down your answer and why
```

Treats empty text as "missing" (usually right for a secret):

```
# practice/env_config_practice.py – Edge cases section
value = os.getenv("OPENAI_API_KEY")
if value is None or value == "":
    raise MissingConfigError("...")
```

Treats empty text as "okay, just empty" (fine for some settings, not usually for a key):

```
# practice/env_config_practice.py – Edge cases section
value = os.getenv("OPENAI_API_KEY")
if value is None:
    raise MissingConfigError("...")
# empty text would pass through here without an error
```

Pick one for your loader and write one sentence saying why. Then check the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
test 1: empty .env file
    run load_config()
    observe: does it raise MissingConfigError, or does it silently continue?
    write down which one happened, and whether that's what you want

test 2: .env contains "OPENAI_API_KEY=" (empty value)
    run load_config()
    observe: is the value None, or an empty string ""?
    decide: should an empty string count as "missing"? write down your answer and
```

If your current `require_env()` only checks `if value is None`, an empty string will slip through as "valid" — is that the behavior you actually want for a secret like an API key?

```
# practice/env_config_practice.py – Edge cases section
def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value
```

What's missing: actual test calls, and the lenient version (which only checks `value is None`, with no `== ""` part). Write both yourself, and try them against an empty `.env` file and a `.env` with `OPENAI_API_KEY=`, then compare against the [Solution](#).

Difference between Basic and Intermediate: same underlying idea (test the 2 textbook cases, then check the code) at 2 completeness levels — Basic and Intermediate test the 2 textbook cases the exercise names, as pseudocode and near-complete code respectively, with Intermediate adding the type contract real Python expects.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)